DEEP LEARNING • FOUNDATIONS

Lecture 00

深度学习基础

从多层感知机到反向传播

从一个可手算的神经网络出发，理解一次训练的全部数学

主题	MLP、Loss、Chain Rule、Backpropagation、PyTorch
适用	已学过机器学习、希望系统复习并进入 CNN 的学习者
配套	Workbook ·Answer Key ·Formula Sheet
数学约定	列向量为单样本，行向量为 PyTorch 的 batch 输入

原创建构讲义·矢量教学图·含逐步数值验证与延伸参考资料

版本 1.0 ·2026-11-12

如何使用这套讲义

第一遍优先顺序是：**00.1–00.5 建模与前向**，**00.6–00.8 损失与反向**，**00.9–00.11 优化与代码**，最后用 00.12–00.13 连接泛化与 CNN。阅读推导时，先遮住下一行并自己写出一个链式法则因子；每节的 Concept Check 在 Workbook 中留有空白，答案集中在单独的 Answer Key。

核心直觉

一条贯穿全文的主线：$x \xrightarrow{W,b} z \xrightarrow{g} a \to \hat{y} \xrightarrow{y} L$。前向传播计算损失；反向传播计算损失对参数的导数；优化器才利用导数修改参数。**这三个动作必须区分。**

Contents

00.1 从机器学习到神经网络 3
- 00.1.1 参数化模型为何需要训练 . . . 3
- 00.1.2 从回归与分类走向神经元 . . . 3

00.2 单个神经元：精确的数学定义 4
- 00.2.1 输入、权重、偏置与激活 . . . 4

00.3 从神经元到多层感知机 MLP 5
- 00.3.1 一层就是一组可并行计算的神经元 . . . 5
- 00.3.2 两层隐藏层的结构与参数 . . . 5
- 00.3.3 Mini-batch 的两种记号不能混用 . . . 5

00.4 激活函数：非线性从哪里来 6
- 00.4.1 线性堆叠为什么可以合并 . . . 6
- 00.4.2 常见函数、导数与使用边界 . . . 6

00.5 前向传播：完整的 2–2–1 手算 7
- 00.5.1 前向计算究竟是什么 . . . 7
- 00.5.2 用 NumPy 复现刚才的算式 . . . 8

00.6 损失函数：让“预测错了多少”可求导 9
- 00.6.1 从预测到损失 . . . 9
- 00.6.2 二分类与多分类损失 . . . 9
- 00.6.3 Logits、Softmax、稳定计算 . . . 9

00.7 梯度与链式法则：反向传播的全部基础 10
- 00.7.1 从普通导数到梯度 . . . 10
- 00.7.2 三次递进：一层、嵌套、分支 . . . 11
- 00.7.3 Jacobian 与向量链式法则 . . . 12

00.8 反向传播：逐参数追踪全部梯度 12

00.8.1 反向传播不等于梯度下降 12
00.8.2 用 00.5 的网络做一次完整反向计算 12
00.8.3 把通用公式与数值结果对齐 14
00.8.4 一次真正可执行的梯度数值检查 14

00.9 梯度下降、Momentum 与 Adam 14
00.9.1 从梯度到参数更新 14
00.9.2 Momentum：把历史梯度方向带进来 15
00.9.3 Adam：一阶矩、二阶矩与偏差校正 16

00.10 Sample、Batch、Epoch、Iteration 16

00.11 PyTorch：把数学翻译成训练循环 17
00.11.1 Tensor、Module、Autograd 分别负责什么 17
00.11.2 完整、可运行的分类 MLP 17
00.11.3 逐行追踪一次训练 18

00.12 训练、验证与泛化 19
00.12.1 三种数据划分有不同职责 19
00.12.2 PyTorch 里的三个模式相关概念 20

00.13 从 MLP 走向 CNN 20
00.13.1 保持训练算法，改变网络结构 20
00.13.2 进入 Lecture 01 前的能力清单 21

学习后检验：五个理解问题 21

附录 A ·推导补完 22

附录 B ·Mini-batch 梯度推导 23

附录 C ·调试清单与实践 24

参考资料与进一步阅读 25

00.1 从机器学习到神经网络

00.1.1 参数化模型为何需要训练

参数化模型（*parametric model*）将一族函数表示为 $f_\theta(x)$。输入 x 是已观察的数据，θ 是模型决定如何处理数据的可调量。以房价回归为例，给定面积 x，预测 $\hat{y} = wx + b$。改变 w, b 就会改变整条预测直线，因此 $\theta = (w, b)$ 构成参数。学习不是给每个输入死记一个答案，而是选择让训练样本总体误差较小的参数。

对样本 $\{(x_i, y_i)\}_{i=1}^N$，先定义单样本损失 $L(f_\theta(x_i), y_i)$，再定义经验风险

$$J(\theta) = \frac{1}{N}\sum_{i=1}^{N} L(f_\theta(x_i), y_i), \qquad \theta^* \in \arg\min_\theta J(\theta). \tag{1}$$

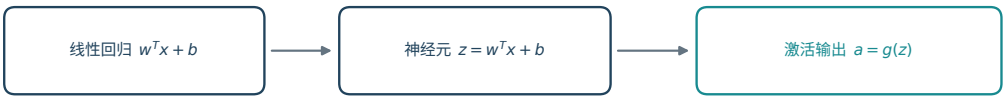
写出 $\arg\min$ 并不是已经找到全局最优：深网络一般是非凸优化问题，实际训练得到的通常是性能足够好的参数。**评估指标**（accuracy 等）用于报告模型好坏；可导的训练**损失函数**则用于产生参数更新方向，两者不必相同。

00.1.2 从回归与分类走向神经元

线性回归直接输出 $w^T x + b$；逻辑回归把同一个值送进 Sigmoid，得到二分类的估计概率：

$$\hat{p} = \text{sigmoid}(w^T x + b), \qquad \text{sigmoid}(z) = \frac{1}{1 + e^{-z}}.$$

从计算形式看，逻辑回归就是带 Sigmoid 的单个神经元。区别不在名字，而在模型的函数结构、目标变量与损失设计。



线性模型可视为恒等激活神经元

单个线性决策函数难以表达 XOR 之类的非线性边界：$(0,0), (1,1)$ 属于 A，而 $(0,1), (1,0)$ 属于 B 时，一条直线无法把两组完全分开。多个神经元并行学习不同特征，隐藏层通过非线性变换组合这些特征，增强函数表达能力。**但神经元数量并不自动保证泛化**；还取决于训练数据、结构、优化与正则化。

Common Mistake

不要把“神经网络更深”直接等同于“训练效果更好”。深度增加表达能力，但也改变优化难度、数据需求和过拟合风险。后面还会证明：若层间没有非线性，无论堆叠多少层，整体依旧可以合并为单个仿射变换。

Concept Check ·暂不提供答案

在 $\hat{y} = w^T x + b$ 中,哪些量由训练改变?为什么不能只用训练集准确率衡量泛化?(Workbook C1)

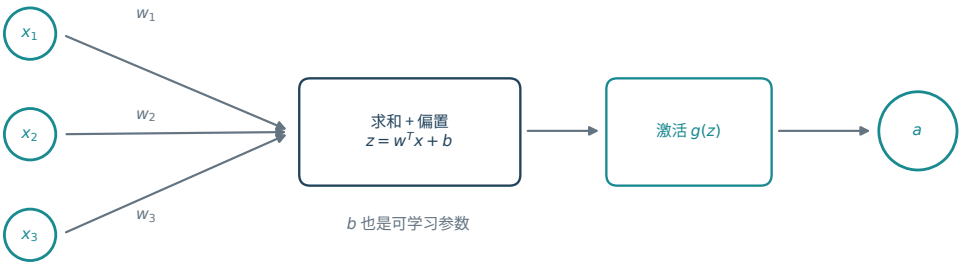
00.2 单个神经元：精确的数学定义

00.2.1 输入、权重、偏置与激活

设输入 $x \in \mathbb{R}^n$ 为列向量，权重 $w \in \mathbb{R}^n$，偏置 $b \in \mathbb{R}$，则

$$z = w^T x + b = \sum_{j=1}^{n} w_j x_j + b \in \mathbb{R}, \qquad a = g(z) \in \mathbb{R}.$$

w_j 控制第 j 个特征对 z 的影响，b 改变没有输入时的截距；激活函数 g 决定输出如何响应。z 通常称 **预激活值**（*pre-activation*），a 称 **激活值**（*activation*）。n 个输入对应 n 个权重和 1 个偏置，总参数数目 $n + 1$。



Worked Example

给定 $x = (2, -1)^T$、$w = (0.5, 0.3)^T$、$b = -0.2$。先算点积 $w^T x = 0.5 \cdot 2 + 0.3 \cdot (-1) = 0.7$,再加偏置得 $z = 0.5$。若选 ReLU,$a = \max(0, 0.5) = 0.5$;若选 Sigmoid,$a = 1/(1+e^{-0.5}) \approx 0.62246$。**同一组参数，激活函数不同，输出不同。**

训练时，参数 w, b 被注册为需要梯度的量，输入数据 x 与真实标签 y 通常固定不变。某些场景会优化输入（如对抗样本），但这不是本课程默认的监督学习流程。

Concept Check ·暂不提供答案

若把 x 从 3 维改为 5 维，一个标量输出神经元需要多少权重和偏置?（Workbook C2、M1）

00.3 从神经元到多层感知机 MLP

00.3.1 一层就是一组可并行计算的神经元

让第 $l-1$ 层有 n_{l-1} 个激活量，第 l 层有 n_l 个神经元。**采用列向量约定**：

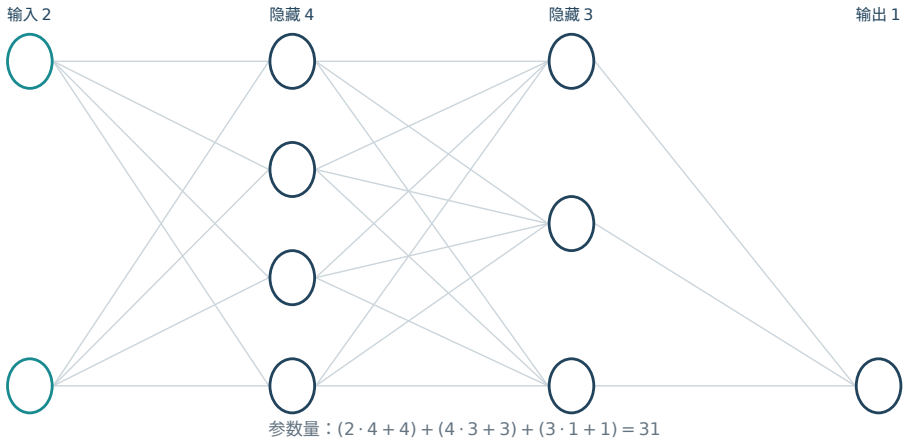
$$a^{[0]} = x \in \mathbb{R}^{n_0}, \tag{2}$$

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}, \quad W^{[l]} \in \mathbb{R}^{n_l \times n_{l-1}},\ b^{[l]} \in \mathbb{R}^{n_l}, \tag{3}$$

$$a^{[l]} = g^{[l]}(z^{[l]}) \in \mathbb{R}^{n_l}. \tag{4}$$

矩阵 $W^{[l]}$ 的第 i 行给出第 i 个新神经元的全部输入权重；内维 n_{l-1} 消去，输出自然是 n_l 维。**全连接层**（*fully connected layer*）指每个输出神经元都读取上一层全部激活值。输入层通常不算一个有权重的层；**宽度**（*width*）是神经元数，**深度**（*depth*）的计数方式因文献而异，必须声明是否把输出层计入。

00.3.2 两层隐藏层的结构与参数



图示 $2 \to 4 \to 3 \to 1$：层间参数为 $4 \cdot 2 + 4 = 12$，$3 \cdot 4 + 3 = 15$，$1 \cdot 3 + 1 = 4$，合计 31。一般公式是

$$\#\theta = \sum_{l=1}^{K} (n_l n_{l-1} + n_l) = \sum_{l=1}^{K} n_l (n_{l-1} + 1). \tag{5}$$

逐层计算实际上是在做函数复合：$f_\theta = g^{[K]} \circ A^{[K]} \circ \cdots \circ g^{[1]} \circ A^{[1]}$，其中 $A^{[l]}(u) = W^{[l]}u + b^{[l]}$。复合函数一旦确定，前向就是按顺序求值，反向就是对复合函数逐层应用链式法则。

00.3.3 Mini-batch 的两种记号不能混用

若一次处理 B 个样本，数学推导可把输入按列堆叠成 $X \in \mathbb{R}^{n_0 \times B}$，于是 $Z^{[l]} = W^{[l]} A^{[l-1]} + b^{[l]} \mathbf{1}_B^T \in \mathbb{R}^{n_l \times B}$。PyTorch 的 `nn.Linear` 通常把样本放在**第一维**，以 $X_{\text{torch}} \in \mathbb{R}^{B \times n_0}$ 输入，内部等价于 $X_{\text{torch}} (W^{[l]})^T + b^{[l]}$。参数本身仍是形状 (n_l, n_{l-1})。

Concept Check ·暂不提供答案

为什么数学表达中的 WX 与 PyTorch 中的 XW^T 不冲突？（Workbook C3、M2）

00.4 激活函数：非线性从哪里来

00.4.1 线性堆叠为什么可以合并

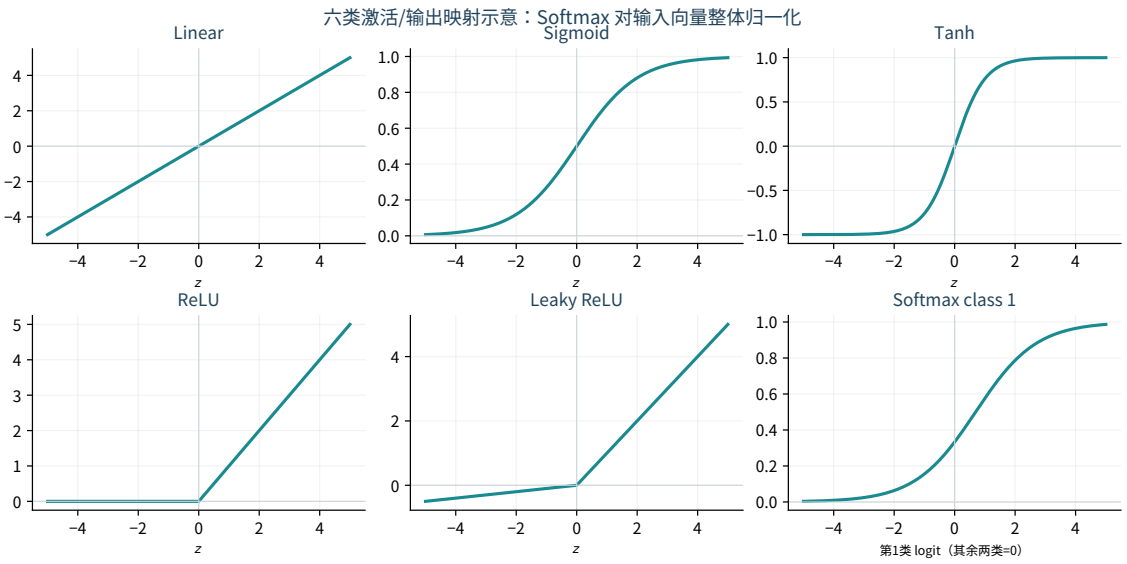
若没有非线性，两层是 $z^{[1]} = W^{[1]}x + b^{[1]}$、$z^{[2]} = W^{[2]}z^{[1]} + b^{[2]}$。代入第一式：

$$z^{[2]} = W^{[2]}(W^{[1]}x + b^{[1]}) + b^{[2]} \tag{6}$$

$$= \underbrace{W^{[2]}W^{[1]}}_{\widetilde{W}}x + \underbrace{W^{[2]}b^{[1]} + b^{[2]}}_{\widetilde{b}}. \tag{7}$$

这仍是 $\widetilde{W}x + \widetilde{b}$，即**仿射变换**（affine map）。即使网络再宽、再深，只要没有非线性，该结构的输入输出关系仍是仿射函数；有了 ReLU 等非线性，网络才可组合成分段线性等丰富的函数。

00.4.2 常见函数、导数与使用边界



函数	定义与导数	典型用途 / 风险
Linear	$g(z)=z,\ g'(z)=1$	回归输出;隐藏层只使用它没有新增非线性
Sigmoid	$\sigma(z)=(1+e^{-z})^{-1},\ \sigma'=\sigma(1-\sigma)$	二分类概率;两侧饱和时导数接近 0
Tanh	$\tanh z,\quad g'=1-\tanh^2 z$	值域 $(-1,1)$;两侧同样饱和
ReLU	$\max(0,z),\ g'=\mathbf{1}_{z>0}$	隐藏层常用;负半轴梯度为 0,可产生失活神经元
Leaky ReLU	$\max(\alpha z,z),\quad g'=\alpha\ (z<0),\ 1\ (z>0)$	负半轴仍传梯度;需选择 $\alpha>0$
Softmax	$p_i=e^{z_i}/\sum_j e^{z_j}$	一组互斥类别的分布;**不是逐元素独立函数**

严格来说，ReLU 在 0 处不可导；实现中选择某个**次梯度**（*subgradient*）约定，PyTorch 在 0 处使用 0。对于 Softmax，导数是**矩阵**而非独立的每元素一维导数：

$$\frac{\partial p_i}{\partial z_j}=p_i(\delta_{ij}-p_j),\qquad J_{\text{softmax}}=\operatorname{diag}(p)-pp^T. \tag{8}$$

其中 $\delta_{ij}=1$（$i=j$）或 0（否则）。某一类 logit 改变，**所有类别的概率**通常都会变化。

核心直觉

Sigmoid 的两端像已经拧到极限的阀门：输入再变化，输出几乎不动，所以梯度极小。ReLU 在正半轴的斜率保持为 1，因此常被用于隐藏层。但在负半轴它也可能彻底停止传递梯度。

Concept Check ·暂不提供答案

把 $z=8$ 增加到 9，Sigmoid 和 ReLU 哪个输出变化更明显？为什么？（Workbook C4）

00.5 前向传播：完整的 2–2–1 手算

00.5.1 前向计算究竟是什么

前向传播（*forward propagation*）就是按计算图从输入到输出**逐步求值**。它既不计算参数梯度，也不会自动更新参数。为后续反向传播，必须清楚保存了哪些输入、权重、预激活值和激活值。

贯穿全书的网络：2 个输入、2 个 ReLU 隐藏神经元、1 个线性输出

定义单样本 $x=\begin{bmatrix}1\\2\end{bmatrix}$、目标 $y=0.4$。参数为

$$W^{[1]}=\begin{bmatrix}0.2&-0.1\\0.4&0.3\end{bmatrix}\ (2\times 2),\quad b^{[1]}=\begin{bmatrix}0.1\\-0.2\end{bmatrix}\ (2\times 1),$$

$$W^{[2]} = \begin{bmatrix} 0.7 & -0.5 \end{bmatrix} \ (1\times 2), \qquad b^{[2]} = 0.05 \ (1\times 1).$$

第 1 步：计算第一个隐藏神经元的加权和：$0.2\cdot 1 + (-0.1)\cdot 2 = 0$，再加偏置 0.1 得 $z_1^{[1]} = 0.1$。
第 2 步：第二个隐藏神经元：$0.4\cdot 1 + 0.3\cdot 2 = 1.0$，加偏置 -0.2 得 $z_2^{[1]} = 0.8$。因此

$$z^{[1]} = W^{[1]}x + b^{[1]} = \begin{bmatrix} 0.1 \\ 0.8 \end{bmatrix}, \quad a^{[1]} = \mathrm{ReLU}(z^{[1]}) = \begin{bmatrix} 0.1 \\ 0.8 \end{bmatrix}.$$

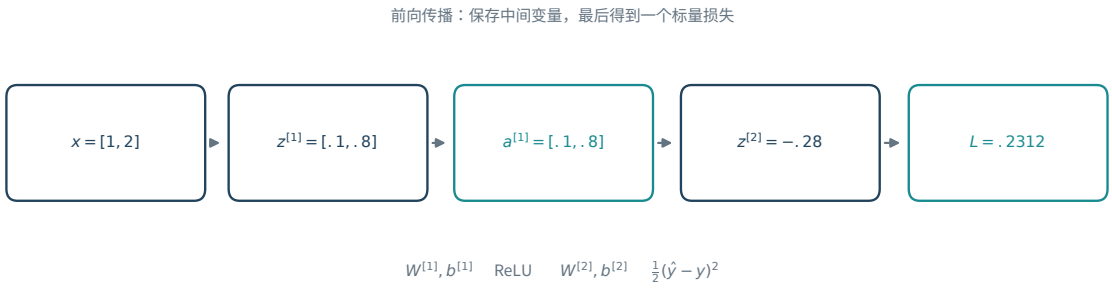
第 3 步：输出层不用 ReLU，直接给回归值：

$$z^{[2]} = W^{[2]}a^{[1]} + b^{[2]} = 0.7(0.1) - 0.5(0.8) + 0.05 = -0.28.$$

第 4 步：令 $\hat{y} = a^{[2]} = z^{[2]} = -0.28$。预测与标签的差为 $\hat{y} - y = -0.68$。本例采用平方损失的一半：

$$\boxed{L = \frac{1}{2}(\hat{y} - y)^2 = \frac{1}{2}(-0.68)^2 = 0.2312.}$$

维度链：$(2\times 2)(2\times 1) + (2\times 1) \to (2\times 1)$，$(1\times 2)(2\times 1) + (1\times 1) \to (1\times 1)$，损失是标量。



00.5.2 用 NumPy 复现刚才的算式

```python
import numpy as np
x = np.array([1., 2.])
W1 = np.array([[.2, -.1], [.4, .3]])
b1 = np.array([.1, -.2])
W2 = np.array([[.7, -.5]])
b2 = np.array([.05])
z1 = W1 @ x + b1 # (2,)
a1 = np.maximum(z1, 0) # (2,)
y_hat = (W2 @ a1 + b2)[0] # scalar
loss = .5 * (y_hat - .4)**2 # 0.2312
```

此处使用 NumPy 一维数组，形状为 `(2,)` 而非严格的 `(2,1)`；在纸上我们仍用列向量表达。当从数学推导切换到实际代码时，要特别检查这种表示差异。

Concept Check ·暂不提供答案

不看答案，手算本例的 $z^{[1]}, a^{[1]}, \hat{y}, L$，并写出每一步的形状。（Workbook M3）

00.6 损失函数：让“预测错了多少”可求导

00.6.1 从预测到损失

真实标签（*ground truth*）为 y，预测为 $\hat{y}$。单样本损失 L 是一个数，多样本目标 J 常取平均。以均方误差为例

$$L_{\text{MSE}} = (\hat{y} - y)^2, \qquad J_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2.$$

本讲义 2–2–1 例题采用 $\frac{1}{2}(\hat{y} - y)^2$，加 $\frac{1}{2}$ 只为求导方便，不改变其最优解。不能把示例中的 $\frac{1}{2}$ 遗漏后仍宣称梯度相同。

00.6.2 二分类与多分类损失

二元交叉熵（*binary cross-entropy, BCE*）对 $p \in (0, 1)$ 的形式为

$$L_{\text{BCE}}(p, y) = -[y \log p + (1 - y) \log(1 - p)], \qquad y \in \{0, 1\}.$$

$y = 1$ 时模型若只给真实类很小的概率，$-\log p$ 很大；给真实类接近 1 的概率，损失接近 0。**互斥多类别交叉熵**（*categorical cross-entropy*）对类别概率 $p \in \mathbb{R}^C$ 与 one-hot 标签 $y \in \{0, 1\}^C$ 为

$$L_{\text{CE}} = -\sum_{c=1}^{C} y_c \log p_c = -\log p_k, \quad \text{若真实类别为} k.$$

CE 正是 one-hot 标签下的**负对数似然**（*negative log-likelihood, NLL*）。

00.6.3 Logits、Softmax、稳定计算

Logits（未归一化类别分数）是最后一层线性输出 $s \in \mathbb{R}^C$，可能为负也不要求总和为 1；Softmax 将其转为类别概率。为数值稳定，计算时通常减去 $m = \max_c s_c$：

$$p_c = \frac{e^{s_c - m}}{\sum_j e^{s_j - m}}, \qquad L = -s_k + \log \sum_j e^{s_j}.$$

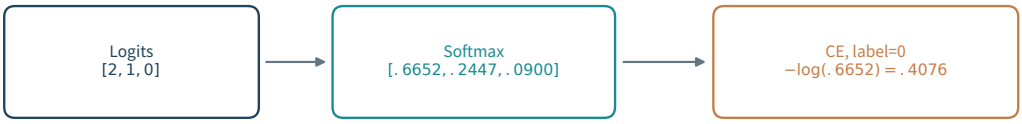
最后一个式子可进一步用 log-sum-exp 稳定实现，$\log \sum_j e^{s_j} = m + \log \sum_j e^{s_j - m}$。

Worked Example

三分类 logits $s = (2, 1, 0)$；真实标签是第 0 类。$e^2 \approx 7.3891, e^1 \approx 2.7183, e^0 = 1$，总和约 11.1073。概率依次为

$$p \approx (0.665241, 0.244728, 0.090031), \qquad L = -\log(0.665241) \approx 0.407606.$$

若把第 0 类概率从 0.665 降为 0.1，其损失将升为约 2.303。**准确率**对两次都预测对的情况可能完全相同，CE 却能区分置信程度并提供有用梯度。



在 PyTorch，互斥多类别分类通常使用 `nn.CrossEntropyLoss`。其**输入直接为 logits**，不要先手工 Softmax。类别索引标签使用 `torch.long` 类型，形状 `(B,)`；logits 形状为 `(B,C)`。

若使用一个 logit 进行二分类,则适合 `BCEWithLogitsLoss`;它同样直接接收 logits。

Common Mistake

对 logits 手动 Softmax 再送给 `CrossEntropyLoss` 是双重处理，得到的数值与梯度不符合预期。分类准确率涉及 `argmax`，对分数的小扰动通常没有梯度；这解释了为什么它适合评估，却不是常规可微训练目标。

Concept Check ·暂不提供答案

三个 logits $(2, 1, 0)$ 若目标从第 0 类改为第 2 类，Softmax 概率是否改变？Loss 呢？（Workbook M4）

00.7 梯度与链式法则：反向传播的全部基础

00.7.1 从普通导数到梯度

一元函数 $y = f(x)$ 在 x 的导数 $f'(x) = \lim_{h\to 0}[f(x+h) - f(x)]/h$，描述局部输出对输入的变化率。多变量函数 $L(w_1, \ldots, w_p)$ 的偏导数 $\partial L/\partial w_j$ 只改变 w_j，其他量暂时固定；把全部偏导组织成列向量就得到**梯度**（*gradient*）$\nabla_w L \in \mathbb{R}^p$。

一阶近似 $L(w+\Delta w) \approx L(w)+\nabla L(w)^T \Delta w$。若 $\Delta w = -\eta \nabla L(w)$,则 $\Delta L \approx -\eta \|\nabla L\|_2^2 \leq 0$；在**足够小的学习率、局部光滑**等条件下，这给出了沿负梯度减小损失的动机。该近似不意味着任意大的学习率都能降低损失。

00.7.2 三次递进：一层、嵌套、分支

例 1：标量一层计算

令 $L(x) = x^2$。用幂函数求导得到 $\mathrm{d}L/\mathrm{d}x = 2x$。当 $x = 3$，斜率为 6；沿负方向微小移动，损失会下降。若 $L(x) = 3x + 1$，则导数恒为 3，意味着移动 1 单位预测局部增加 3 单位。

例 2：两层复合函数

令 $u = g(x) = 2x - 1$、$L = f(u) = u^2$，即 $L(x) = (2x-1)^2$。链式法则是

$$\frac{\mathrm{d}L}{\mathrm{d}x} = \underbrace{\frac{\mathrm{d}L}{\mathrm{d}u}}_{2u}\underbrace{\frac{\mathrm{d}u}{\mathrm{d}x}}_{2} = 4(2x-1).$$

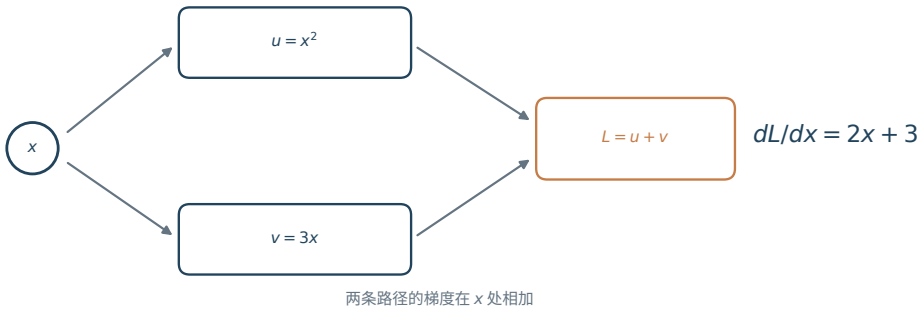
若 $x = 2$，先得到 $u = 3$，再得 $\mathrm{d}L/\mathrm{d}u = 6$ 与 $\mathrm{d}u/\mathrm{d}x = 2$，故总导数 12。这里每个因子对应图上一条局部边。

例 3：多路径汇合

同一参数 x 分两条路径作用于损失：$u = x^2$、$v = 3x$、$L = u + v$。则

$$\frac{\mathrm{d}L}{\mathrm{d}x} = \underbrace{\frac{\partial L}{\partial u}\frac{\mathrm{d}u}{\mathrm{d}x}}_{1\cdot 2x} + \underbrace{\frac{\partial L}{\partial v}\frac{\mathrm{d}v}{\mathrm{d}x}}_{1\cdot 3} = 2x + 3.$$

当 $x = 2$，两条路径分别贡献 4、3，合计 7。**只保留其中一条路径会漏掉梯度**。



00.7.3 Jacobian 与向量链式法则

对于向量函数 $z = f(x)$，$x \in \mathbb{R}^n, z \in \mathbb{R}^m$，Jacobian 是 $J_f \in \mathbb{R}^{m\times n}$，其中 $(J_f)_{ij} = \partial z_i/\partial x_j$。若最终 L 是标量，用**列梯度**约定就有

$$\nabla_x L = J_f(x)^T \nabla_z L \qquad [(n\times m)(m\times 1) \to (n\times 1)]. \tag{9}$$

对线性层 $z = Wx + b$，Jacobian 就是 W，所以反传到输入的梯度是 $W^T\nabla_z L$。这也是矩阵反向公式里转置的来源。

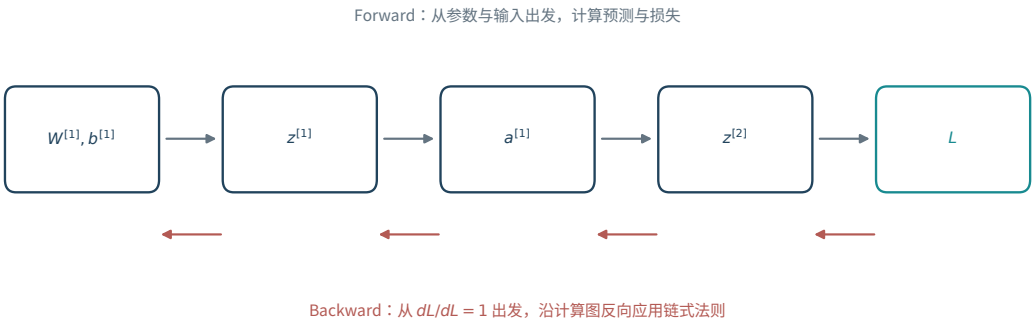
Concept Check ·暂不提供答案

若 $L = u + v, u = x^2, v = 3x$，为什么不能只反传通过 u 的梯度？（Workbook C5、D1）

00.8 反向传播：逐参数追踪全部梯度

00.8.1 反向传播不等于梯度下降

反向传播（*backpropagation*）是对已经构建的计算图应用链式法则、**高效计算各参数梯度**的过程；**梯度下降**（*gradient descent*）是在梯度已知之后，按规则调整参数的方法。因为 L 位于计算图终点，反向从 $\partial L/\partial L = 1$ 开始，逐步乘上局部导数，复用中间梯度，避免对每个参数都重复展开整个函数。



00.8.2 用 00.5 的网络做一次完整反向计算

仍使用 $x = (1, 2)^T$、$y = 0.4$、$z^{[1]} = a^{[1]} = (0.1, 0.8)^T$、$\hat{y} = -0.28$、$L = 0.2312$。统一用符号

$$\delta^{[l]} := \nabla_{z^{[l]}} L \in \mathbb{R}^{n_l}.$$

第 0 步：从损失出发。输出层是恒等激活 $a^{[2]} = z^{[2]}$，且 $L = \frac{1}{2}(\hat{y} - y)^2$。因此

$$\frac{\partial L}{\partial \hat{y}} = \frac{1}{2}\cdot 2(\hat{y} - y) = \hat{y} - y = -0.68, \qquad \delta^{[2]} = -0.68.$$

这一步的两个局部因子是 $\partial L/\partial \hat{y} = -0.68$ 和 $\partial \hat{y}/\partial z^{[2]} = 1$。所以 $\delta^{[2]} = (-0.68)\cdot 1$。

第 1 步：求输出层参数梯度。 $z^{[2]} = w_{21}a_1^{[1]} + w_{22}a_2^{[1]} + b^{[2]}$。由于 $\partial z^{[2]}/\partial w_{2j} = a_j^{[1]}$，$\partial z^{[2]}/\partial b^{[2]} = 1$，所以

$$\frac{\partial L}{\partial w_{21}} = \underbrace{\delta^{[2]}}_{-0.68}\underbrace{a_1^{[1]}}_{0.1} = -0.068, \tag{10}$$

$$\frac{\partial L}{\partial w_{22}} = (-0.68)(0.8) = -0.544, \tag{11}$$

$$\nabla_{W^{[2]}} L = \delta^{[2]}(a^{[1]})^T = \begin{bmatrix} -0.068 & -0.544 \end{bmatrix} \ (1 \times 2), \tag{12}$$

$$\nabla_{b^{[2]}} L = \delta^{[2]} = -0.68 \ (1 \times 1). \tag{13}$$

注意这里的权重梯度依赖**前向保存的激活值**。只有知道当时的 $a^{[1]}$，才能算出梯度。

第 2 步：误差向隐藏层传播。从输出节点到隐藏激活，$\nabla_{a^{[1]}} L = (W^{[2]})^T \delta^{[2]}$：

$$\nabla_{a^{[1]}} L = \begin{bmatrix} 0.7 \\ -0.5 \end{bmatrix} (-0.68) = \begin{bmatrix} -0.476 \\ 0.340 \end{bmatrix} \quad (2 \times 1).$$

隐藏层的 $z_1^{[1]} = 0.1 > 0$、$z_2^{[1]} = 0.8 > 0$，故 ReLU 在这两个位置的局部导数都为 1。使用逐元素乘法 $\odot$：

$$\delta^{[1]} = \underbrace{(W^{[2]})^T \delta^{[2]}}_{\nabla_{a^{[1]}} L} \odot \underbrace{g'(z^{[1]})}_{(1,1)^T} = \begin{bmatrix} -0.476 \\ 0.340 \end{bmatrix}.$$

如果某个 $z_j^{[1]} < 0$，其 ReLU 局部导数为 0，从该神经元往前传的梯度就会被截断。此例没有这种截断。

第 3 步：求隐藏层的四个权重与两个偏置梯度。因为 $z_i^{[1]} = \sum_j W_{ij}^{[1]} x_j + b_i^{[1]}$，逐项有 $\partial L/\partial W_{ij}^{[1]} = \delta_i^{[1]} x_j$：

$$\nabla_{W^{[1]}} L = \delta^{[1]} x^T = \begin{bmatrix} -0.476 \\ 0.340 \end{bmatrix} \begin{bmatrix} 1 & 2 \end{bmatrix} = \boxed{\begin{bmatrix} -0.476 & -0.952 \\ 0.340 & 0.680 \end{bmatrix}} \ (2 \times 2), \tag{14}$$

$$\nabla_{b^{[1]}} L = \delta^{[1]} = \boxed{\begin{bmatrix} -0.476 \\ 0.340 \end{bmatrix}} \ (2 \times 1). \tag{15}$$

输入也能收到梯度，但**常规训练不更新输入**：$\nabla_x L = (W^{[1]})^T \delta^{[1]} = (0.0408, 0.1496)^T$。这一项属于计算图的可导性，不意味着输入是可学习参数。

00.8.3 把通用公式与数值结果对齐

对于 $l = K, K-1, \ldots, 1$ 的前向 $z^{[l]} = W^{[l]}a^{[l-1]} + b^{[l]}$，反向递推为

$$\nabla_{W^{[l]}} L = \delta^{[l]}(a^{[l-1]})^T \qquad (n_l \times n_{l-1}), \tag{16}$$

$$\nabla_{b^{[l]}} L = \delta^{[l]} \qquad (n_l \times 1), \tag{17}$$

$$\delta^{[l-1]} = \left((W^{[l]})^T \delta^{[l]}\right) \odot g^{[l-1]'}(z^{[l-1]}) \qquad (n_{l-1} \times 1). \tag{18}$$

其中最后一行只用于 $l - 1 \geq 1$ 的隐藏层；到输入为止不需要再乘尚不存在的 $g^{[0]'}$。若一参数影响损失的路径不止一条，则**从多个子节点传回的梯度要先求和**，再继续传给这个参数。共享参数的 CNN 在后续课程尤其需要此规则。

对 B 个样本的平均损失 $J = \frac{1}{B}\sum_i L_i$，梯度是每个样本梯度的平均。以列堆叠的 $A \in \mathbb{R}^{n\times B}$、$\Delta \in \mathbb{R}^{m\times B}$ 表示各样本 δ（未含平均系数），则 $\nabla_W J = \Delta A^T/B$、$\nabla_b J = \Delta \mathbf{1}_B/B$。若 Δ 已包含 $1/B$，就不能重复除以 B。

00.8.4 一次真正可执行的梯度数值检查

有限差分（*finite difference*）比较解析导数与相邻损失值：

$$\frac{\partial L}{\partial \theta_j} \approx \frac{L(\theta_j + h) - L(\theta_j - h)}{2h}, \qquad h = 10^{-6}.$$

以 $W^{[1]}_{12} = -0.1$ 为例，理论上 $\partial L/\partial W^{[1]}_{12} = -0.952$。只改该参数为 -0.1 ± 10^{-6}，其他参数不变，计算两次前向损失并做差分；数值应接近 -0.952。配套 `source/numeric_checks.py` 实际检查了 $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}$ 的全部 9 个参数，最大绝对误差约为 5.5×10^{-11}（当前 Python/NumPy 环境所得）。这不是靠图像“看起来相近”作出的验证。

Common Mistake

梯度核验是局部的。 ReLU 在 0 处不可导；在不光滑边界附近，有限差分与某个次梯度约定可能不一致。若比较数值梯度，先确认扰动没有跨过激活状态变化的拐点。

Concept Check ·暂不提供答案

从 $\delta^{[2]} = -0.68$ 推导出 $\nabla_{W^{[1]}} L$；每一行都写明形状。解释为何 `backward()` 后参数还没变。（Workbook D2、M5、C6）

00.9 梯度下降、Momentum 与 Adam

00.9.1 从梯度到参数更新

求出梯度不等于完成训练。最基本的**梯度下降**（*gradient descent*）是

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta L_t, \qquad \eta > 0.$$

若使用全部训练数据计算 ∇J，是 full-batch gradient descent；若用每个小批次的均值梯度来更新，常称 mini-batch SGD。SGD 的 “stochastic” 来自样本批次的随机性，不代表每次更新只能用 1 个样本。

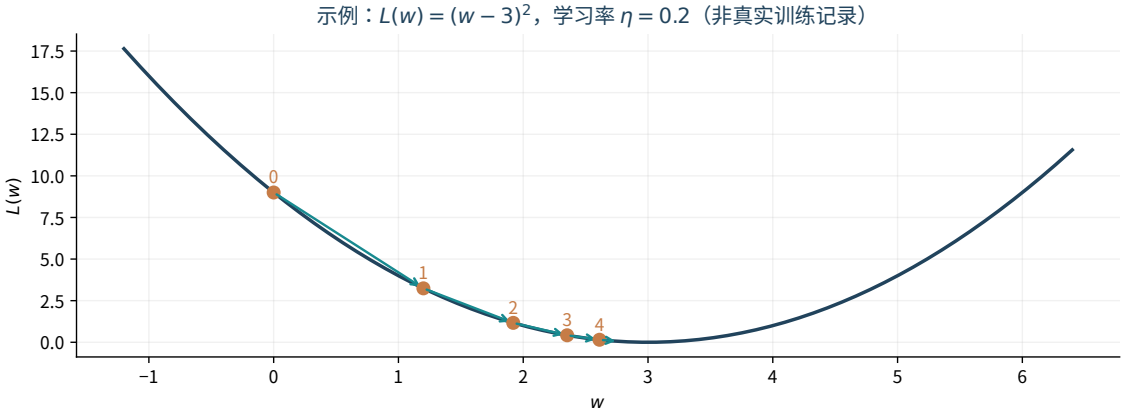
Worked Example

对上面的网络，一次学习率 $\eta = 0.1$ 的 SGD 更新：

$$W^{[1]}_{\text{new}} = \begin{bmatrix} .2 & -.1 \\ .4 & .3 \end{bmatrix} - .1 \begin{bmatrix} -.476 & -.952 \\ .340 & .680 \end{bmatrix} = \begin{bmatrix} .2476 & -.0048 \\ .3660 & .2320 \end{bmatrix},$$

$$b^{[1]}_{\text{new}} = \begin{bmatrix} .1476 \\ -.2340 \end{bmatrix}, \quad W^{[2]}_{\text{new}} = \begin{bmatrix} .7068 & -.4456 \end{bmatrix}, \quad b^{[2]}_{\text{new}} = .118.$$

这是**先统一计算所有原始参数的梯度，然后统一更新**；不要边算一层梯度边更新上一层参数，否则后面反传用的就不是同一个前向计算图。



图中的曲线 $L(w) = (w-3)^2$ 是单变量的**教学示例**，演示学习率 0.2 沿负梯度从 0 走向 3，不是神经网络真实训练曲线。

00.9.2 Momentum：把历史梯度方向带进来

动量法（*momentum*）用指数滑动平均减弱方向振荡，例如

$$v_t = \beta v_{t-1} + (1-\beta) g_t, \qquad \theta_t = \theta_{t-1} - \eta v_t.$$

这里 g_t 是本批次梯度，β 典型取接近 1 的值。与把 $(1-\beta)$ 吸收到学习率中的其他常见写法相比，数值尺度不同；比较不同教材时不能只看 β 而忽略学习率定义。动量直觉像小球在坡面滚动：同方向多次出现的梯度更能累积影响，交替变号的分量会相互抵消。

00.9.3 Adam：一阶矩、二阶矩与偏差校正

Adam（*adaptive moment estimation*）同时追踪梯度 g_t 的一阶矩与平方梯度的二阶矩：

$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \tag{19}$$

$$v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^{\odot 2}, \tag{20}$$

$$\widehat{m}_t = \frac{m_t}{1-\beta_1^t}, \quad \widehat{v}_t = \frac{v_t}{1-\beta_2^t}, \tag{21}$$

$$\theta_t = \theta_{t-1} - \eta \frac{\widehat{m}_t}{\sqrt{\widehat{v}_t} + \varepsilon}. \tag{22}$$

$m_0 = v_0 = 0$。为什么要校正？假设前几步梯度几乎固定为 g，则 $m_1 = (1-\beta_1)g$，$v_1 = (1-\beta_2)g^2$：初始化为 0 使早期估计系统性偏小。若取 $\beta_1 = 0.9$，第一步 $m_1 = 0.1g$，校正后 $\widehat{m}_1 = g$。对二阶矩同理。不同参数坐标的 $\widehat{v}_t$ 不一样，因而 Adam 的**有效步长可逐坐标不同**。ε 防止分母为 0；Adam 并非总优于 SGD，泛化与超参数仍需验证。

Concept Check ·暂不提供答案

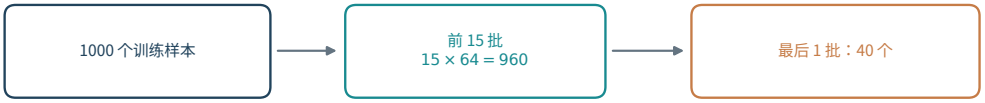
Adam 若省略一阶、二阶矩的偏差校正，刚开始为什么会产生失真？（Workbook C7、D3）

00.10 Sample、Batch、Epoch、Iteration

样本（*sample*）是一个 (x_i, y_i)；**数据集**（*dataset*）是样本集合；**批大小**（*batch size*）是每次用于计算梯度的样本数；**迭代**（*iteration/step*）通常指一次参数更新；**轮次**（*epoch*）通常指把训练集遍历一遍。实际计数还会受到数据增强、梯度累积、特殊采样策略的影响，本节只讨论标准数据加载器。

Worked Example

训练集 $N = 1000$，$B = 64$。前 15 个完整批次包含 $15 \cdot 64 = 960$ 个样本，最后还剩 40 个样本。因此 $\lceil 1000/64 \rceil = 16$ 批；若**保留**最后不足 64 的批，1 Epoch 就做 16 次更新；若 `drop_last=True`，只做 15 次更新，40 个样本本轮未处理（跨轮是否总会遗漏取决于 shuffle 等设置）。



保留最后一批：每个 Epoch 16 次更新；丢弃最后一批：15 次更新

将 Batch Size 从 64 改到 100（完整处理 1000 样本）每 Epoch 更新次数从 16 降到 10。但不能因此断言大 Batch 一定更慢或更快：GPU 并行效率、梯度方差、学习率与总训练步数都会影响实际效果。**不要把 Epoch 当成一次梯度更新**。

Concept Check ·暂不提供答案

在 1000 个样本的训练集中，$B = 64$ 与 `drop_last=True` 分别会有多少个 batch 和更新？（Workbook C8）

00.11 PyTorch：把数学翻译成训练循环

00.11.1 Tensor、Module、Autograd 分别负责什么

`torch.Tensor` 是具有 dtype、device、shape 的多维数组；`requires_grad=True` 表示希望对其参与的运算记录梯度。`nn.Module` 管理模型子层与参数；`nn.Linear(in,out)` 管理形状 (out, in) 的权重与形状 $(out,)$ 的偏置，`nn.ReLU()` 则逐元素应用 ReLU。**自动微分**（*autograd*）记录运算产生的依赖关系，通过链式法则把损失梯度传回需要梯度的叶节点参数。

必须区分 `model.parameters()`（可训练参数对象）、`param.grad`（当前累积的梯度存储）与 `optimizer.step()`（真正修改参数）。PyTorch 默认的 `backward()` 会**累积**叶参数梯度，所以训练前通常调用 `optimizer.zero_grad()`。

00.11.2 完整、可运行的分类 MLP

下面是无需下载数据、只依赖 PyTorch 的小型 XOR 风格分类练习。这是**机制演示代码**而非学术性能基准。文件原版在 `source/minimal_training.py`:

```
import torch
from torch import nn

torch.manual_seed(7)
x = torch.tensor([[0., 0.], [0., 1.],
                  [1., 0.], [1., 1.]], dtype=torch.float32)
y = torch.tensor([0, 1, 1, 0], dtype=torch.long)

class TinyMLP(nn.Module):
   def __init__(self):
      super().__init__()
      self.net = nn.Sequential(
         nn.Linear(2, 16), nn.ReLU(), nn.Linear(16, 2)
      )

   def forward(self, x):
      return self.net(x)

model = TinyMLP()
criterion = nn.CrossEntropyLoss()
```

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.03)

for epoch in range(300):
   model.train()
   optimizer.zero_grad()
   logits = model(x) # (4, 2), NOT probabilities
   loss = criterion(logits, y) # scalar
   loss.backward()
   if epoch == 0:
      print('weight grad:', model.net[0].weight.grad.shape)
   optimizer.step()

model.eval()
with torch.no_grad():
   predicted = model(x).argmax(dim=1)
print('predicted:', predicted.tolist())
```

x 形状是 $(4, 2)$，y 形状 $(4,)$ 且 dtype 是 long，*logits* 形状是 $(4, 2)$，标量 *loss* 可以直接 `backward()`。`nn.Linear(2,16)` 的权重梯度形状为 $(16, 2)$。单层 ReLU 不保证任何随机种子下必然学会 XOR；此处展示代码及可观测机制，不把某一学习曲线当成普遍规律。

00.11.3 逐行追踪一次训练

```
optimizer.zero_grad() # clear previous gradient buffers
logits = model(x) # forward: build differentiable graph
loss = criterion(logits, y) # calculate scalar training objective
loss.backward() # chain rule, write to parameter.grad
optimizer.step() # update model parameters
```

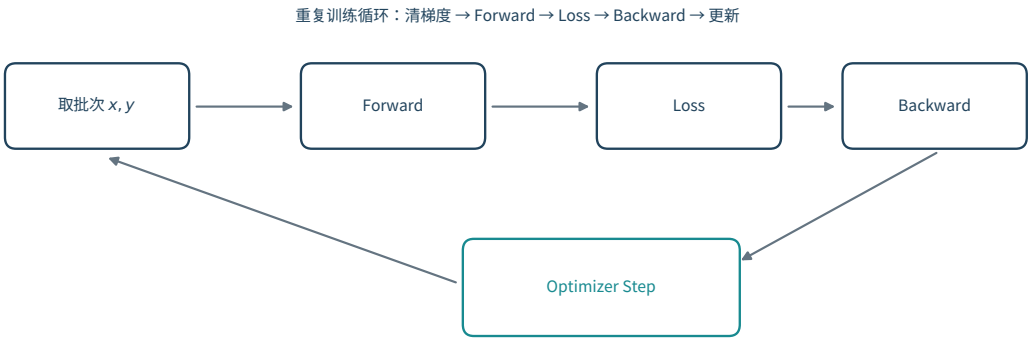
第一行清理**之前**的梯度存储，避免错误叠加；第二行 `model(x)` 在 Python 中调用模块的 `forward` 并返回 logits；第三行用标签将预测变成标量损失 $L(\theta)$；第四行沿计算图反传，计算 $\partial L/\partial\theta$ 并累积到参数 `.grad`；第五行才按 Adam 或 SGD 等规则调整参数。

因此只调用 `model(x)` 产生的是**推理输出**而非训练；只执行 `loss.backward()` 也只是算出梯度，参数值保持不变。需要诊断时可查看

```
print(model.net[0].weight.shape) # torch.Size([16, 2])
print(model.net[0].weight.grad.shape) # torch.Size([16, 2])
```

Common Mistake

常见错误：标签写成 float 而给 `CrossEntropyLoss` 的类别索引模式使用；输出只有 1 个 logit 却希望预测 2 类并用相同目标形状；忘记清梯度；把 `model.eval()` 当成禁用梯度；把 `torch.no_grad()` 与切换训练模式混为一谈。



Concept Check ·暂不提供答案

从计算图角度说明每一行训练代码的职责；为什么不能把 `step()` 放在 `backward()` 之前？（Workbook P1、P2）

00.12 训练、验证与泛化

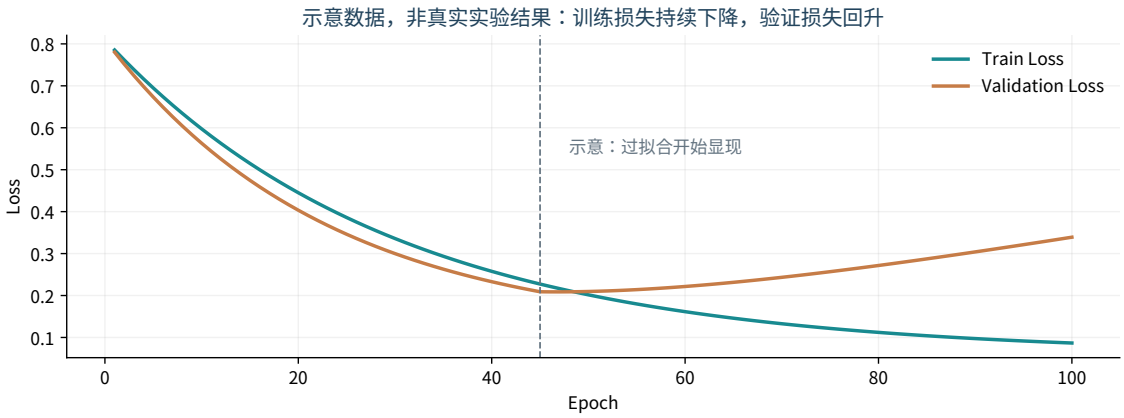
00.12.1 三种数据划分有不同职责

训练集（*training set*）用于计算梯度并更新参数；**验证集**（*validation set*）用于开发期间比较超参数、结构与早停等策略，**通常不用于通过梯度直接更新模型**；**测试集**（*test set*）尽量留到模型选择完成后作最终泛化评估。若反复在测试集上调参，测试集就不再是独立的最后检验。

欠拟合（*underfitting*）是训练集本身也表现很差，可能是模型能力、特征、训练时长或优化不足；**过拟合**（*overfitting*）指训练样本表现很好，遇到未见样本却明显退化。**偏差/方差**（*bias/variance*）是理解误差来源的概念框架，不能机械等同于两条损失曲线的某个固定差值。

Worked Example

若训练准确率为 100%，验证准确率只有 68.75%，这是显著的**泛化差距**，值得检查是否过拟合、数据量不足、训练/验证分布不同以及标签错误；仅凭这两个数值不能唯一确诊原因。先检查划分方式、数据泄漏、类别分布，随后尝试模型容量、正则化、数据增强等调整，并用新验证结果判断是否改善。



00.12.2 PyTorch 里的三个模式相关概念

`model.train()` 开启训练模式，主要影响 Dropout、BatchNorm 等具有模式依赖行为的层；`model.eval()` 切换到评估模式，影响这些层的行为，但**不会单独禁用自动微分**。`with torch.no_grad():` 则不记录需要梯度的运算图，可减少验证/推理开销。标准评估写法：

```
model.eval()
with torch.no_grad():
   val_logits = model(val_x)
   val_loss = criterion(val_logits, val_y)
```

验证完继续训练前再调用 `model.train()`。如果没有 Dropout/BatchNorm 等层，简单 MLP 的 train/eval 数值可能相同；但正确的工程习惯仍应保留模式切换。

Concept Check ·暂不提供答案

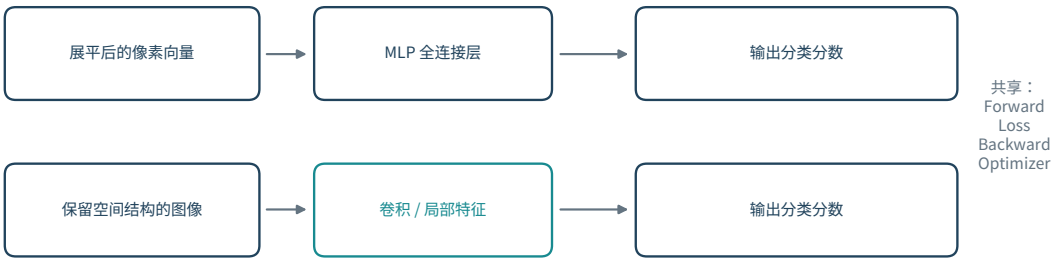
若执行 `model.eval()` 但未使用 `no_grad()`，是否一定没有计算图？（Workbook C9、A1）

00.13 从 MLP 走向 CNN

00.13.1 保持训练算法，改变网络结构

图像 $H \times W \times C$ 展平后可以交给 MLP，但一个全连接层 $HWC \to m$ 需要 $m(HWC+1)$ 个参数；例如 $32 \times 32 \times 3 \to 128$ 需要 $128(3072+1) = 393344$ 个参数。它也不会在网络结构上主动利用邻近像素的空间关联。

CNN 引入局部连接与**参数共享**（*parameter sharing*）：同一卷积核参数在不同空间位置重复使用。把卷积看作另一种可微计算模块，**Forward、Loss、Backward、Optimizer 的训练闭环丝毫没有消失**。下一讲才详细回答卷积核、stride、padding、输出特征图形状、反向传播与卷积实现。



00.13.2 进入 Lecture 01 前的能力清单

- **张量维度**：能够从 (B, d) 追踪 `nn.Linear(d,h)` 到 (B, h)，并解释数学列向量与框架 batch 约定的差异。
- **Forward**：不查答案，手算本讲义 2–2–1 的 $z, a, \hat{y}, L$。
- **Loss**：知道二分类、多分类使用什么形式的 logit 和标签；会算 Softmax–CE。
- **Backward**：能独立推导至少两个参数梯度，理解转置、逐元素乘法和多路径梯度相加。
- **优化器**：明确 `backward()` 计算梯度，`step()` 更新参数，`zero_grad()` 清理梯度。
- **PyTorch**：能检查 tensor shape 和 dtype，读懂一轮训练与验证，理解模式切换和自动微分控制。

这些能力分别对应 **Lab 00: Tensor & Autograd** 和 **Lab 01: MLP & Backpropagation** 的准备工作；本套讲义不会替你完成实验任务。若仍不能独立解释 $\delta^{[1]} = (W^{[2]})^T \delta^{[2]} \odot g'(z^{[1]})$，先返回 00.7–00.8，而不是直接开始 CNN。

学习后检验：五个不允许只背术语的问题

1. 若两个线性层之间没有 ReLU，增加中间神经元为什么仍不保证增加输入输出函数的非线性?
2. 对一个 $(B, 2)$ 的输入送入 `nn.Linear(2,3)`，权重为什么是 $(3, 2)$，而输出为什么是 $(B, 3)$?
3. 同一变量通过两个分支流向 L 时，为什么每一条路径都必须计算贡献?
4. 某层 `weight.grad` 已有正确值却发现参数没变，最可能缺了哪一步？能否用数值比较验证?
5. 如果训练准确率 100% 而验证准确率 69%，先检查什么数据问题？为什么不能立刻断言模型一定过大?

附录 A ·推导补完：矩阵求导、Softmax–CE 与 BCE

A.1 从元素偏导理解为什么权重梯度是外积

考虑最常见的线性层 $z = Wx + b$，其中 $z \in \mathbb{R}^m$、$x \in \mathbb{R}^n$、$W \in \mathbb{R}^{m\times n}$。不使用矩阵求导记忆公式，而是先写第 i 个输出分量：

$$z_i = \sum_{j=1}^{n} W_{ij}x_j + b_i.$$

给定上一节点传回的梯度 $\delta_i = \partial L/\partial z_i$，对权重分量 W_{pq} 求导。由于 W_{pq} 只在 z_p 中出现，而且其系数为 x_q，

$$\frac{\partial z_i}{\partial W_{pq}} = \begin{cases} x_q, & i = p \\ 0, & i \neq p. \end{cases}$$

当下游损失依赖全部 z_i 时，沿所有中间输出汇总：

$$\frac{\partial L}{\partial W_{pq}} = \sum_{i=1}^{m} \frac{\partial L}{\partial z_i}\frac{\partial z_i}{\partial W_{pq}} = \delta_p x_q.$$

矩阵 δx^T 的 (p, q) 元恰好是 $\delta_p x_q$，因而

$$\boxed{\nabla_W L = \delta x^T \in \mathbb{R}^{m\times n}}.$$

对偏置同理：$\partial z_i/\partial b_p = \mathbf{1}_{i=p}$，所以 $\partial L/\partial b_p = \delta_p$。对输入 x_q 则每个输出 z_i 都依赖 x_q：

$$\frac{\partial L}{\partial x_q} = \sum_{i=1}^{m} \delta_i W_{iq}, \quad \text{即} \quad \boxed{\nabla_x L = W^T\delta}.$$

这样三条线性层反向公式都可从元素偏导独立推导，不需要凭借图形形状猜测转置该放在哪一边。

A.2 交叉熵对 Logits 的导数为何如此简单

Softmax 的输入是 $s \in \mathbb{R}^C$,预测分布为 $p_i = e^{s_i}/\sum_j e^{s_j}$。真实类别为 k,交叉熵 $L = -\log p_k$。直接把 Softmax 代入损失：

$$L = -\log\left(\frac{e^{s_k}}{\sum_j e^{s_j}}\right) = -s_k + \log\sum_{j=1}^{C} e^{s_j}.$$

对任意第 i 个 logit 求导，第一项 $-s_k$ 的导数为 $-\mathbf{1}_{i=k}$；第二项用链式法则：

$$\frac{\partial}{\partial s_i}\log\sum_j e^{s_j} = \frac{1}{\sum_j e^{s_j}}\frac{\partial}{\partial s_i}\sum_j e^{s_j} = \frac{e^{s_i}}{\sum_j e^{s_j}} = p_i.$$

因此我们得到最常用的结果：

$$\boxed{\frac{\partial L}{\partial s_i} = p_i - \mathbf{1}_{i=k}; \qquad \nabla_s L = p - y_{\text{onehot}}.} \tag{23}$$

对三分类例题 $s = (2, 1, 0)$、$k = 0$，该梯度为

$$\nabla_s L \approx (-0.334759,\ 0.244728,\ 0.090031)^T.$$

第 0 类梯度为负，按负梯度更新有上调该类 logit 的趋势；另外两类梯度为正，按负梯度更新有下调它们的趋势。三项相加约为 0，这是因为所有 logits 同时增加同一常数不会改变 Softmax 概率。

A.3 对二分类 Logit 直接求梯度

当网络输出单个 logit s，定义 $p = \sigma(s)$ 和二元交叉熵

$$L = -y \log p - (1 - y) \log(1 - p), \qquad y \in \{0, 1\}.$$

先对 p 求导：$\partial L/\partial p = -y/p + (1 - y)/(1 - p)$。再乘 $\partial p/\partial s = p(1 - p)$，

$$\frac{\partial L}{\partial s} = \left(-\frac{y}{p} + \frac{1 - y}{1 - p}\right) p(1 - p) \tag{24}$$

$$= -y(1 - p) + (1 - y)p = -y + yp + p - yp = \boxed{p - y}. \tag{25}$$

这与多分类交叉熵的 $p - y$ 形式相似，但输入输出张量与损失定义不同。

`BCEWithLogitsLoss` 将 Sigmoid 与 BCE 的计算组合以提升数值稳定性。在二分类输出为 $(B, 1)$ 时，float 标签通常也要是 $(B, 1)$；多分类 CE 的类别索引标签则用 $(B,)$ 的整数。不要把两种设置混为一谈。

附录 B ·批量反向：从单样本推广到真正训练

B.1 为什么批量梯度是逐样本梯度的平均

最简单的训练目标是 $J = \frac{1}{B} \sum_{i=1}^{B} L_i$。对一个共享参数 θ 求导，利用求和与求导的线性性质：

$$\nabla_\theta J = \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta L_i.$$

每个样本都使用同一套权重。因此一个 batch 不是创建 B 组各自独立的参数，而是每个样本贡献一份梯度，然后按损失的 reduction 规则聚合。

以列堆叠的 $A \in \mathbb{R}^{d\times B}$ 为例，第 i 列为输入 a_i。第 l 层误差信号组成 $\Delta = [\delta_1, \dots, \delta_B] \in \mathbb{R}^{m\times B}$。单样本 $\nabla_W L_i = \delta_i a_i^T$，求和得

$$\sum_i \delta_i a_i^T = \begin{bmatrix} \delta_1 & \cdots & \delta_B \end{bmatrix} \begin{bmatrix} a_1^T \\ \vdots \\ a_B^T \end{bmatrix} = \Delta A^T.$$

故 $\nabla_W J = \Delta A^T/B$，形状 $(m\times B)(B\times d)\to(m\times d)$；$\nabla_b J = \Delta \mathbf{1}_B/B$。数学推导中的 A 按列堆叠，PyTorch 则一般按行堆叠，因此在框架内部相应乘法方向也随之调整。

Common Mistake

如果 `nn.CrossEntropyLoss(reduction='mean')` 已返回平均损失，自动微分也会自动包含 $1/B$。**不要手工再次将参数梯度除以 batch size**。若使用 `reduction='sum'`，则得到的是各样本损失求和的梯度。

B.2 小试验：梯度累积到底会发生什么

用同一模型、同一批数据连续计算两次 `backward()`，假如不调用 `zero_grad()`，则参数的 `.grad` 是两次反传所得的梯度和；如果损失和参数都未变化，第二次反传后的梯度约为第一次的两倍。反过来说，有时为了模拟大 batch，会**故意累积多个 micro-batch 梯度**，但那需要正确归一化，并推迟到累积完成后才执行 `optimizer.step()`。本课程的基础示例采用每轮清理的方法。

```
# one-step diagnostic only: no optimizer.step() between calls
optimizer.zero_grad()
loss = criterion(model(x), y)
loss.backward()
g1 = model.net[0].weight.grad.detach().clone()
# regenerate graph for a fresh forward
loss2 = criterion(model(x), y)
loss2.backward()
g2 = model.net[0].weight.grad.detach().clone()
print(torch.allclose(g2, 2 * g1)) # usually True here
```

必须重建计算图：默认情况下，不能对同一次已释放的普通前向图反复执行 `backward()`，因此上例第二次重新计算 `model(x)`。这体现了自动微分记录实际执行的前向运算，不是存储某个永远不变的静态数学公式。

附录 C ·从报错里找出理解漏洞

C.1 五个实际容易出现的错误

1. **矩阵内维不匹配：** 若输入 X 是 $(32,5)$ 却设置 `nn.Linear(4,3)`，其 weight 形状 $(3,4)$，无法计算 $(32,5)\times(4,3)$。修改输入特征数或层定义，不能靠随机加 `.reshape` 掩盖数据语义错误。

2. **分类输出/目标不匹配：**对多分类 `CrossEntropyLoss`，输出一般为 (B, C)，类别索引标签为 $(B,)$ long；不能因为分类有 3 类就把每个标签写成 $(B, 3)$ 的随意整数矩阵。

3. **过早 Softmax：**预测输出可以在分析阶段用 `softmax(logits)` 展示概率；但训练计算 `CrossEntropyLoss` 应使用 logits。

4. **忘记清梯度：**若没有指定梯度累积策略，前一 batch 的梯度混入当前 update。排查时打印 `weight.grad.norm()`，对比是否随迭代异常增大，但数值增长不一定**仅**由未清零导致。

5. **验证阶段仍使用训练模式：** Dropout 与 BatchNorm 的行为可能导致验证分数不稳定或不同。用 `model.eval()` 与 `no_grad()` 配合，并在返回训练前 `model.train()`。

C.2 在 iPad 上进行 45 分钟的自检

第 1–10 分钟：独立画出 $2 \to 2 \to 1$ 网络和张量形状，不看讲义写出 Forward；第 11–25 分钟：从 L 反传至每一项 W, b，用不同颜色标注梯度流向；第 26–35 分钟：检查 $\delta_1 = (W_2)^T \delta_2 \odot g'(z_1)$ 的每个因子为什么出现；第 36–45 分钟：只写训练循环的 5 行核心代码，并解释 `backward()` 与 `step()` 的差别。

合格标准：不看公式表，能在 15 分钟内复原数字 $L = 0.2312$ 和全部 9 个参数的梯度结构；不要求死记四位小数，但至少能独立算出 $\partial L/\partial W_{11}^{[2]} = -0.068$ 和 $\partial L/\partial W_{12}^{[1]} = -0.952$，并完整解释路径。若失败，返回相应步骤重新推导，而不是重复默读整章。

C.3 CNN 前的一个简短思考实验

一张 RGB 图片 $32 \times 32 \times 3$ 共有 3072 个像素值。MLP 全连接到 16 维需要 $16(3072+1) = 49168$ 个参数；CNN 用 16 个 3×3 卷积核，每个核覆盖 3 个输入通道，若每个输出通道各有 1 个偏置，**卷积层参数数目**为

$$16(3 \cdot 3 \cdot 3 + 1) = \boxed{448}.$$

两者是不同的模型结构，输出张量形状和感受野也不同，不能把 448 个参数直接等同于“必然更好的图像识别效果”。但是 448 与 49168 的数量级差异揭示了参数共享的实际意义。具体卷积输出尺寸、stride 与 padding 的分析，将是 Lecture 01 的核心内容。

参考资料与进一步阅读

1. Stanford CS231n, https://cs231n.stanford.edu/; 课程安排中的 Neural Networks & Backpropagation、Optimization 主题，https://cs231n.stanford.edu/schedule.html。

2. Stanford CS230 课程资源与学习指南，https://stanford.edu/~shervine/teaching/cs-230/。
3. Dive into Deep Learning 中文版（MLP、自动微分、优化算法），https://zh.d2l.ai/。
4. PyTorch 官方教程：Autograd，https://docs.pytorch.org/tutorials/beginner/basics/autogradqs_tutorial.html。
5. PyTorch `CrossEntropyLoss` 说明（确认输入是 logits 与类别索引标签类型），https://docs.pytorch.org/docs/stable/generated/torch.nn.CrossEntropyLoss.html。

课程结构与公式以这些权威资料为参考，叙述、数值例题及 13 张矢量教学图由本讲义独立编写、计算与绘制。PyTorch 文档会随版本更新；使用不同版本时应以本机安装版本的官方 API 说明为准。